

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automatics

Mihail Pancenco FAF-243

Report

Laboratory work No. 10

Operating Systems

Add Your Own System Call: trace

Supervisors:
Dumitraşcu Marius, University assistant
Reiţman Patricia, University assistant
DISA, FCIM, UTM

Chişinău – 2026

Table of Contents

1 Introduction **3**

2 Tasks and results **3**

2.1 Part 1: The number and user side 3

2.2 Part 2: Kernel handling and return values 4

2.3 Part 3: Select syscalls with a mask 5

2.4 Optional: sysinfo 6

3 Conclusions **7**

1 Introduction

This laboratory adds a per-process syscall tracer to xv6. The kernel was built and booted under QEMU 10.2.1 in Ubuntu 26.04.1 LTS on VirtualBox on 9 November 2026. An isolated Git worktree from the installed xv6 revision `06aad25` preserved the preceding laboratory files.

2 Tasks and results

2.1 Part 1: The number and user side

The original kernel was built and booted first. Syscall 22 was already assigned to `sync`, so `trace` received number 23.

```
misa@UbuntuVM:os-labs-09-10-20261109$ cd xv6-riscv
misa@UbuntuVM:xv6-riscv$ git rev-parse --short HEAD; qemu-system-riscv64 --version | head -1
06aad25
QEMU emulator version 10.2.1 (Debian 1:10.2.1+ds-1ubuntu3.2)
misa@UbuntuVM:xv6-riscv$ make -j4 kernel/kernel fs.img > ../logs/build-baseline.txt 2>&1 && echo 'Build OK'
Build OK
misa@UbuntuVM:xv6-riscv$ make qemu CPUS=1
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -global virtio-m

xv6 kernel is booting

init: starting sh
$ echo baseline boot OK
baseline boot OK
$
```

Figure 2.1 - Baseline xv6 boots and starts its shell.

The number goes in `kernel/syscall.h`. The declaration in `user/user.h` and the `entry("trace")` in `user/usys.pl` produce a callable stub. The disassembly confirmed `li a7,23`, `ecall` and `ret`. The Makefile places `user/trace.c` on the disk through `UPROGS`. The launcher sets its own mask and replaces its image with the requested program using `exec`.

```
misa@UbuntuVM:xv6-riscv$ python3 ../stage.py user
user changes applied
misa@UbuntuVM:xv6-riscv$ make -j4 kernel/kernel fs.img > ../logs/build-user.txt 2>&1 && echo 'Build OK'
Build OK
misa@UbuntuVM:xv6-riscv$ riscv64-linux-gnu-objdump -d user/_trace | sed -n '/<trace>:/,+3p'
00000000000003c8 <trace>:
 3c8:   48dd                    li      a7,23
 3ca:   00000073                ecall
 3ce:   8082                    ret
misa@UbuntuVM:xv6-riscv$ make qemu CPUS=1
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -global virt

xv6 kernel is booting

init: starting sh
$ trace 32 grep hello README
3 trace: unknown sys call 23
trace: failed
$
```

Figure 2.2 - Before registration, the kernel receives 23 and returns an unknown-syscall error.

The initial error is expected: the stub enters the kernel correctly, but the dispatch table has no handler for 23 yet.

2.2 Part 2: Kernel handling and return values

The syscall dispatcher indexes its handler table with the number saved from `a7`. The new `sys_trace` handler reads argument 0 from the trapframe via `argint`, stores it in the process's unsigned `trace_mask`, and returns zero. A new process slot starts with a zero mask, and `kfork` copies the parent's mask to the child before it can run. `exec` retains the same process structure and its mask.

After a registered syscall returns, the dispatcher saves its result to `a0`. If that syscall's bit is set, `printk` prints the PID, syscall name and signed result. The dispatch-table bounds check and the additional bit-width check keep the mask shift valid. Non-returning handlers such as `exit` cannot reach the after-call print.

```
p->trapframe->a0 = syscalls[num]();
if (num < 8 * sizeof(p->trace_mask) &&
    (p->trace_mask & (1U << num)) && syscallnames[num])
  printk("%d: syscall %s -> %ld\n", p->pid,
         syscallnames[num], p->trapframe->a0);
```

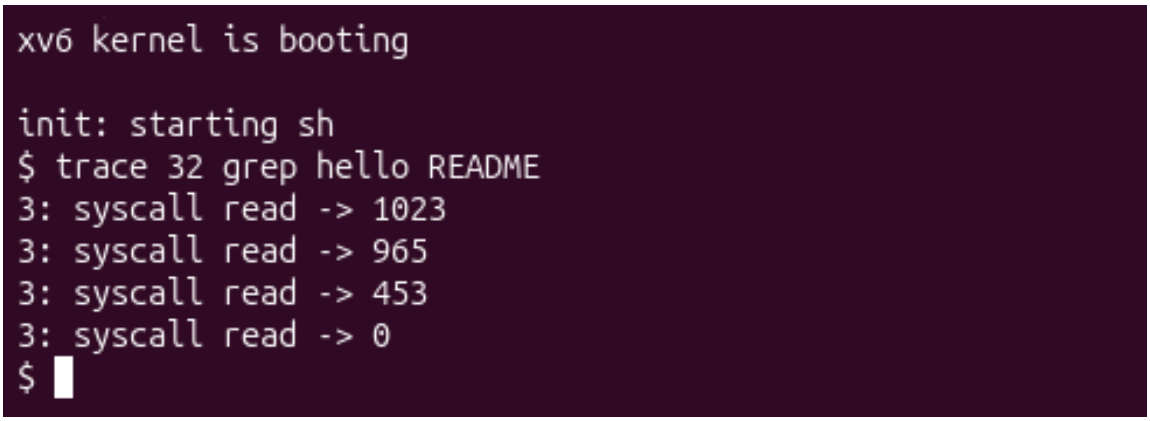


Figure 2.3 - After registration, mask 32 prints the four actual read results.

The positive reads were **1023, 965 and 453 bytes**, totalling **2441 bytes**. The final zero is end of file. Mask 32 selects bit 5 (`SYS_read`), so open and close still execute but their lines are not printed.

2.3 Part 3: Select syscalls with a mask

```
$ trace 2147483647 grep hello README
3: syscall trace -> 0
3: syscall exec -> 3
3: syscall open -> 3
3: syscall read -> 1023
3: syscall read -> 965
3: syscall read -> 453
3: syscall read -> 0
3: syscall close -> 0
$ trace 65536 echo hi
hi4: syscall write -> 2

4: syscall write -> 1
$ trace 2129920 grep hello README
5: syscall open -> 3
5: syscall close -> 0
$ trace 0 echo untraced
untraced
$ echo ordinary
ordinary
$ trace
usage: trace mask command [args...]
$
```

Figure 2.4 - All supported call bits, write-only, open/close, and untraced commands.

Mask	Bit(s)	Observed result
32	5	Only `read`.
2147483647	0–30	Returning calls made by the launcher and `grep`.
2	1	Only `fork`, including in descendant processes.
65536	16	Two writes for `echo hi`: 2 bytes, then 1 byte.
2129920	15, 21	`open` and `close` only.
0	none	No trace lines.

`echo hi` writes `hi` and the newline in two separate calls. The kernel logs the first after `hi` has reached the console, then logs the second after the newline. For open and close, the mask is $2^{15} + 2^{21} = 32768 + 2097152 = \mathbf{2129920}$. The observed run produced only those two call names.

2.3.1 Fork inheritance

The line `np->trace_mask = p->trace_mask;` in `kfork` makes the mask available to the children. In `forkforkfork`, PIDs 9, 10 and 11 printed fork traces. The final `fork` returned -1 when the process table filled, and the test reported `ALL TESTS PASSED`. The figure shows the beginning and end of the saved trace.

```
misa@UbuntuVM:xv6-riscv$ head -9 ../logs/forkforkfork.txt; tail -6 ../logs/forkforkfork.txt
trace 2 usertests forkforkfork
usertests starting
test forkforkfork: 9: syscall fork -> 10
10: syscall fork -> 11
11: syscall fork -> 12
11: syscall fork -> 13
11: syscall fork -> 14
11: syscall fork -> 15
11: syscall fork -> 16
11: syscall fork -> 68
11: syscall fork -> 69
11: syscall fork -> 70
11: syscall fork -> -1
OK
ALL TESTS PASSED
misa@UbuntuVM:xv6-riscv$
```

Figure 2.5 - Three tracing PIDs, process-table exhaustion, and the passing fork test.

2.4 Optional: sysinfo

Syscall 24 fills a `struct sysinfo` with the free-list size in bytes and the number of process slots in use. The allocator helper traverses its list under the allocator lock, and the process helper reads each slot with its own lock held. The handler transfers the two measurements to user space with `copyout`. The standalone command showed **133251072 bytes** free and **3 processes in use**. Mask 2^{24} recorded `sysinfo -> 0`. The original read trace still passed afterward.

```
$ sysinfo
Free memory: 133251072 bytes
Processes in use: 3
$ trace 16777216 sysinfo
4: syscall sysinfo -> 0
Free memory: 133251072 bytes
Processes in use: 3
$ trace 32 grep hello README
5: syscall read -> 1023
5: syscall read -> 965
5: syscall read -> 453
5: syscall read -> 0
$
```

Figure 2.6 - Optional sysinfo, its trace entry and the read-mask regression check.

3 Conclusions

The staged experiment followed syscall 23 from the user stub to the kernel dispatcher, then verified per-process filtering across `exec` and `fork`. The read counts matched the README size, the fork test passed, and the optional call returned its data through the user address space. The submission contains the full xv6 source tree and the terminal results.

Bibliography

[1] Operating Systems, FCIM / FAF, Technical University of Moldova. Laboratory work No. 10: *Add Your Own System Call: trace*. Course handout, 2026–2027.

[2] MIT PDOS. *xv6-riscv*, base revision `06aad25c735fd3159bdfae5680be4eab7b1668b2`.